

GIT AND GITHUB · 45 MINUTES

Git and GitHub

How we build together. Why and how do we share changes?

Arif Solmaz
İSTÜN Mechatronics Engineering



Changing projects and personal memory

The latest file does not explain how we got here.

- Code changes; the product behaves differently.
- An experiment changes; the explanation of its results changes.
- The team changes; people carrying the knowledge move on.

Working alone: find your way back

Your future self will work on this project too.

Remember

Revisit what you changed and why.

Compare

Examine the difference between a working state and a new one.

Continue

Use a recorded state as a new starting point.

Which file should we continue from?

THE PROBLEM

Which one is the latest?

23:47

Robot project

.ino

robot.ino

.ino

robot_v2.ino

.ino

robot_v2_
deniz.ino

.ino

robot_final.ino

.ino

robot_final_
FINAL.ino

.ino

robot_final_
FINAL_kaan.ino

.ino

robot_REALLY_
final.ino

?

Robot team

K

Kaan

Elif, can you send the latest code?

E

Elif

Sent robot_final.ino

robot_final.ino

D

Deniz

But I fixed the sensor in robot_v2...

K

Kaan

The robot left the line again. Which one is the latest??

E

Elif

No idea. Should we try them one by one?

Who changed what? Which one worked? Nobody knows.

As copies multiply, it becomes harder to know which state to trust.

What does version control record?

Different states of our work and the relationships between them.

- Recorded states
- Changes between those states
- Author details and descriptions of changes

Git: a tool for project history

Repository: the files and the records that track them.

Project files

The code, text and related files you work on.

Recorded history

Records you can examine to see how the project changed.

You do not need a GitHub account to use Git.

GitHub: discuss changes together

A platform for sharing Git repositories and discussing contributions.

Git

Keeps a history of changes on your computer.

GitHub

Provides a shared place to publish, discuss and review contributions.

Git can be used without GitHub; other platforms are available.

Local work and sharing

A local copy of history lets you work without a constant connection.

- Inspect history in your local copy.
- Make new records while offline.
- Share your work to bring it together with other contributions.

Commit: a record with a description

Record a particular state of the project and explain the change.

Recorded state

A particular state of the project is kept.

Description

The purpose of the change becomes visible.

Connection

The record connects to the earlier history.

What does history establish?

It shows what changed. Correctness still needs to be checked.

History shows

Which files changed, how they changed and what description was written.

Also needs checking

Working code, valid measurements and sound decisions.

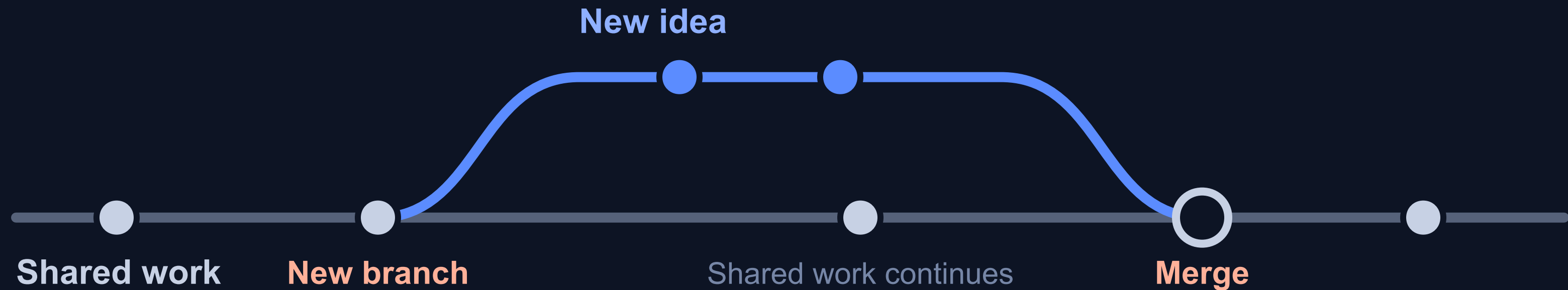
Small changes, understandable decisions

A clear purpose makes a record easier to review and build on.

- Keep related changes in the same record.
- Explain why the change was needed as well as what changed.
- Include enough context for a future reader.

Branch: a separate path of work

Branch: move in a different direction from the same history.



- An idea develops on a separate branch.
- The team brings it into shared work when it is suitable.

A branch gives an experiment room; its result still needs checking.

Merge: bring work together

Merge: bring separate paths into a shared history.

- Independent changes can often merge automatically.
- Competing changes may need a choice.
- No merge conflict does not prove that the system works correctly.

In a team: make progress in parallel

Independent work becomes useful when it leads to shared decisions.

Parallel work

Different tasks can move forward at the same time.

Shared decisions

Evaluate a contribution by its purpose and effects.

Shared memory

Find the discussion alongside the relevant change.

How do people use this day to day?

Follow how a need becomes part of the shared project.

1. The team explains the need and expected result.
2. Someone prepares a change on a separate branch.
3. The team reviews the proposal and the evidence.
4. The team brings a suitable change into the shared project.

GitHub Flow is one example of this way of working.

Issue: a need we can discuss

3 BENEFIT

Your team's to-do list

E Elif's screen

github.com/elif/my-robot/issues

elif / my-robot Public

<> Code Issues 2 Pull requests 0 Actions

Q is:issue state:open Labels New issue

2 Open ✓ 1 Closed Author ▾ · Labels ▾ · Assignee ▾

Take robot photos for the report report #3 opened 2 minutes ago by elif

Motor driver gets hot bug #1 opened 2 days ago by kaan

FOR YOU

Everyone sees who's doing what

✓ Tasks don't vanish in the group chat; you can find them months later.

✓ Every task has an owner.

✓ Capstone project, club, assignment group: it fits them all.

Issue = a task to do or a note about a bug.

When it's done, **Close issue** : it leaves the list, the record stays.

An issue is a shared place to discuss and track a problem or proposal.

Pull request: propose a change

PR: “Should we bring this work into the shared project?”

Proposal

What changed and why?

Discussion

Questions, comments and requests for revision.

Decision

Bring it in, leave it waiting or close the proposal.

Review: a second pair of eyes

Review: understand and evaluate a proposed change.

- Can I understand the purpose and assumptions?
- Does it affect another part of the project?
- Is it clear how the result was checked?

Matplotlib: even where an explanation belongs is discussed together.

History, review and testing do different jobs

They complement one another; each answers a different question.

History

What changed?

Review

Does this decision make sense?

Test or experiment

Do we observe the behavior we expected?

In a student project: a shared hand-in

The team continues from a shared project instead of collecting copies.

- Software, explanations and related text share the same context.
- The hand-in refers to a known recorded state.
- The team can see missing work and proposed improvements.

Commit count does not measure a person's effort or the quality of the work.

In research: keep a trail of the method

Make clear which code and settings produced a result.

- A recorded state of the analysis code and settings.
- Reasons for changes in the method.
- Information about data, the environment and steps to repeat.

In a company: share changes with checks

One person's decision can affect the rest of the product.

Different roles

Development, testing, maintenance and product needs.

Shared decisions

Accept changes after considering their effects.

Continuity

Know which state of the product is in use.

In open source: work with people you have never met

Contributing involves more than writing code.

- Reporting problems and improving explanations are contributions too.
- Fork: a related repository copy under your own account.
- Submitting a proposal does not guarantee acceptance.

Arduino Servo: user observation → issue → proposed documentation fix.

BENEFIT 3 • HANDOVER

Hand the project over to the next person

Help the next person find the files and understand the decisions.

- The next person finds the reasons for decisions alongside the files.
- The project depends less on its original contributor being constantly available.



README: the entrance to a project

Help a first-time visitor know where to start.

- What does this project do?
- What do I need to get started?
- How can I use or try it?
- Where can I find its limits and get help?

MDN Web Docs: making an explanation clearer is a contribution too.

Working culture and shared responsibility

Clear reasoning and respectful communication matter alongside the files.

- Make expectations and responsibilities clear.
- Discuss the change rather than criticize the person.
- Keep the reasons behind important decisions.

The cost: learning and keeping records

It may feel slower at first.

Effort today

Learn concepts, write explanations and organize records.

Value later

Understand history, separate changes and continue the work.

The cost: coordination and waiting

Review can improve quality while adding time to the process.

- Comments and revisions take time.
- Separate work may need decisions when it comes together.
- Match the process to team size and project risk.
- Work tracking and automation on the platform also need maintenance.

Visibility: set boundaries when sharing

A public repository makes the work visible to everyone.

Value of visibility

People can examine, discover and share the work.

Responsibility when sharing

Respect boundaries for confidential information, personal data and access keys.

Deleting the current file does not automatically erase published history.

Public repositories and licensing

A license explains the terms for reuse.

- Visibility: being able to access the content.
- License: terms for use, modification and distribution.
- Contribution rules: how to take part in the project.

Which problems does Git leave to us?

Keeping history addresses one part of an engineering project.

- Document hardware and the environment separately.
- Large files and CAD may need other methods.
- Use suitable tests and backups alongside history.

Additional storage and tools may add costs.

Does every task need the same process?

Choose the tool and process to fit the project.

Value grows

When changes, collaboration, recovery and handover matter.

Keep the process simple

Start with basic records; add suitable rules as the project grows.

Three questions for your next project

Rethink how you work before learning every Git feature.

- Which state of our project works?
- Why did we make this change?
- Could someone continue without us?

Questions and discussion

Which information gets lost most often in your own work?

- How are Git and GitHub different?
- What is the value when working alone?
- What do we gain and spend when sharing?

Four essential terms

Distinguish the concepts before learning every detail.

- Repository: the tracked project and its history.
- Commit: a particular recorded state with a description.
- Branch: a separate path from the same history.
- Pull request / PR: a proposal to bring changes into shared work.

What can a README answer?

The starting information a first-time reader needs.

- Purpose: what does it do and who is it for?
- Getting started: the environment and preparation required.
- Use: expected results and an example.
- Limits: known problems, help and licensing.

A change proposal people can understand

Give the reviewer enough context to make a decision.

- Need: which problem or proposal?
- Change: what was done and what is the scope?
- Checking: what was tried and what was observed?
- Open question: any known limit or pending decision?

Choose according to the project's needs

Consider history, collaboration and file types together.

History needs

Do we need to revisit an earlier state or compare changes?

People's needs

Will others work together or take over the project?

File needs

Mostly code and text, or large data and specialized design files?

Four useful working habits

A routine you can sustain makes the tool more useful.

- Group related changes into readable records.
- Explain the reason for a future reader.
- Review shared changes in proportion to the need.
- Keep the starting instructions current.

Sources and where to go next

Return to these sources and real projects when you need them.

[Pro Git: the basic approach](#)

[GitHub Docs: GitHub and collaboration](#)

[Arduino Servo: a user's contribution](#)

[Matplotlib: reviewing a change together](#)

[MDN: a small documentation contribution](#)